

第二门课 改善深层神经网络：超参数调试、正则化以及优化(Improving Deep Neural Networks:Hyperparameter tuning, Regularization and Optimization)

第一周：深度学习的实践层面(Practical aspects of Deep Learning)

第二门课 改善深层神经网络：超参数调试、正则化以及优化(Improving Deep Neural Networks:Hyperparameter tuning, Regularization and Optimization)

第一周：深度学习的实践层面(Practical aspects of Deep Learning)

- 1.1 训练，验证，测试集 (Train / Dev / Test sets)
- 1.2 偏差，方差 (Bias /Variance)
- 1.3 机器学习基础 (Basic Recipe for Machine Learning)
- 1.4 正则化 (Regularization)
- 1.5 为什么正则化有利于预防过拟合呢？ (Why regularization reduces overfitting?)
- 1.6 dropout 正则化 (Dropout Regularization)
- 1.7 理解 dropout (Understanding Dropout)
- 1.8 其他正则化方法 (Other regularization methods)
- 1.9 归一化输入 (Normalizing inputs)
- 1.10 梯度消失/梯度爆炸 (Vanishing / Exploding gradients)
- 1.11 神经网络的权重初始化 (Weight Initialization for Deep NetworksVanishing / Exploding gradients)
- 1.12 梯度的数值逼近 (Numerical approximation of gradients)
- 1.13 梯度检验 (Gradient checking)
- 1.14 梯度检验应用的注意事项 (Gradient Checking Implementation Notes)

1.1 训练，验证，测试集 (Train / Dev / Test sets)



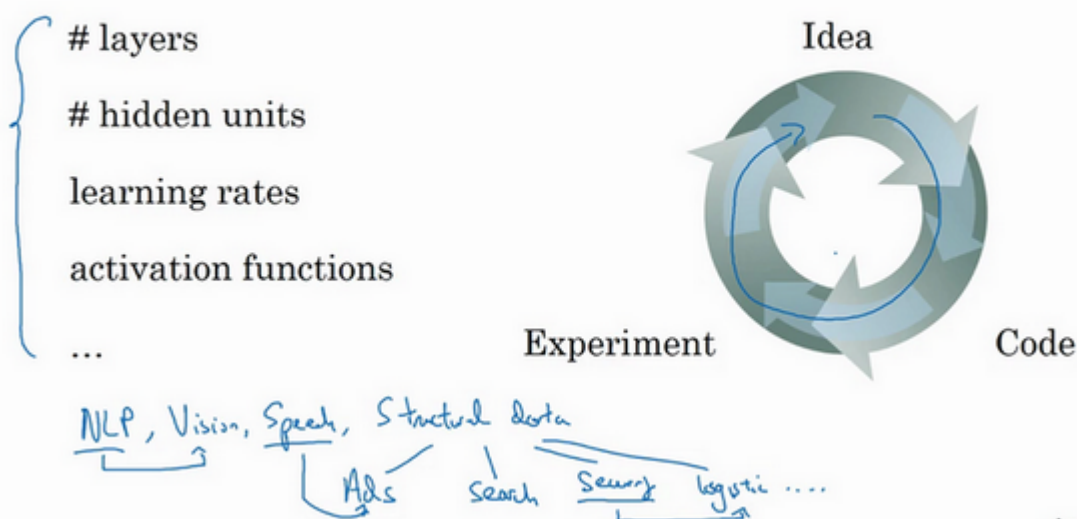
大家可能已经了解了，那么本周，我们将继续学习如何有效运作神经网络，内容涉及超参数调优，如何构建数据，以及如何确保优化算法快速运行，从而使学习算法在合理时间内完成自我学习。

第一周，我们首先说说神经网络机器学习中的问题，然后是随机神经网络，还会学习一些确保神经网络正确运行的技巧，带着这些问题，我们开始今天的课程。

在配置训练、验证和测试数据集的过程中做出正确决策会在很大程度上帮助大家创建高效的神经网络。训练神经网络时，我们需要做出很多决策，例如：

1. 神经网络分多少层
2. 每层含有多少个隐藏单元
3. 学习速率是多少
4. 各层采用哪些激活函数

Applied ML is a highly iterative process

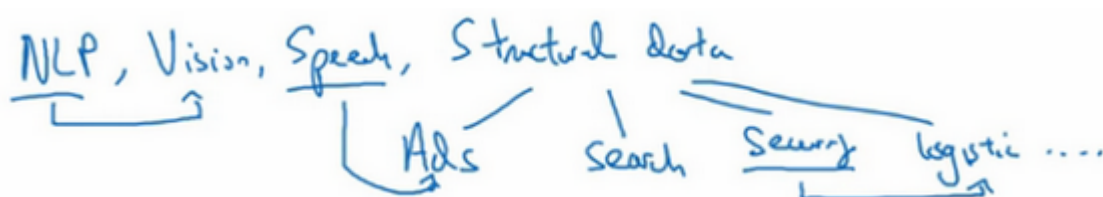


Andrew Ng

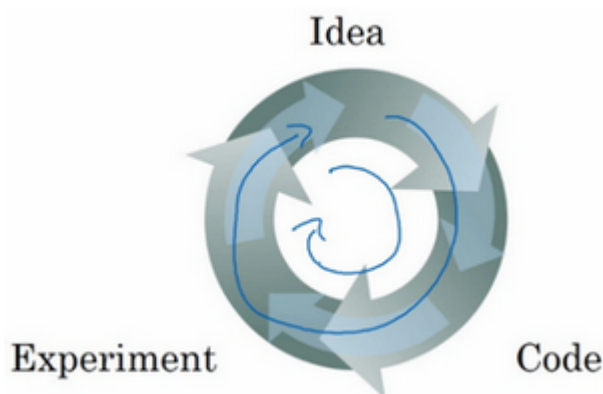
创建新应用的过程中，我们不可能从一开始就准确预测出这些信息和其他超级参数。实际上，应用型机器学习是一个高度迭代的过程，通常在项目启动时，我们会先有一个初步想法，比如构建一个含有特定层数，隐藏单元数量或数据集个数等等的神经网络，然后编码，并尝试运行这些代码，通过运行和测试得到该神经网络或这些配置信息的运行结果，你可能会根据输出结果重新完善自己的想法，改变策略，或者为了找到更好的神经网络不断迭代更新自己的方案。

现如今，深度学习已经在自然语言处理，计算机视觉，语音识别以及结构化数据应用等众多领域取得巨大成功。结构化数据无所不包，从广告到网络搜索。其中网络搜索不仅包括网络搜索引擎，还包括购物网站，从所有根据搜索栏词条传输结果的网站。再到计算机安全，物流，比如判断司机去哪接送货，范围之广，不胜枚举。

我发现，可能有自然语言处理方面的人才想踏足计算机视觉领域，或者经验丰富的语音识别专家想投身广告行业，又或者，有的人想从电脑安全领域跳到物流行业，在我看来，从一个领域或者应用领域得来的直觉经验，通常无法转移到其他应用领域，最佳决策取决于你所拥有的数据量，计算机配置中输入特征的数量，用GPU训练还是CPU，GPU和CPU的具体配置以及其他诸多因素。



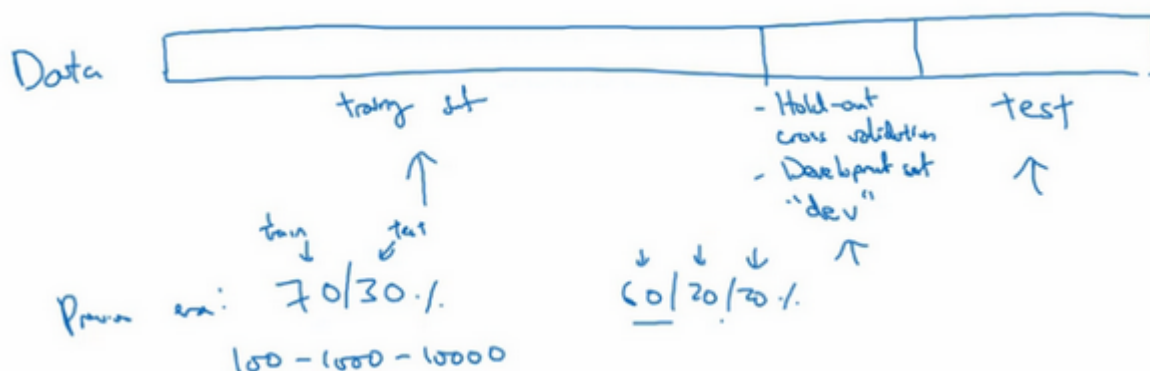
目前为止，我觉得，对于很多应用系统，即使是经验丰富的深度学习行家也不太可能一开始就预设出最匹配的超级参数，所以说，应用深度学习是一个典型的迭代过程，需要多次循环往复，才能为应用程序找到一个称心的神经网络，因此循环该过程的效率是决定项目进展速度的一个关键因素，而创建高质量的训练数据集，验证集和测试集也有助于提高循环效率。



假设这是训练数据，我用一个长方形表示，我们通常会将这些数据划分成几部分，一部分作为训练集，一部分作为简单交叉验证集，有时也称之为验证集，方便起见，我就叫它验证集（**dev set**），其实都是同一个概念，最后一部分则作为测试集。

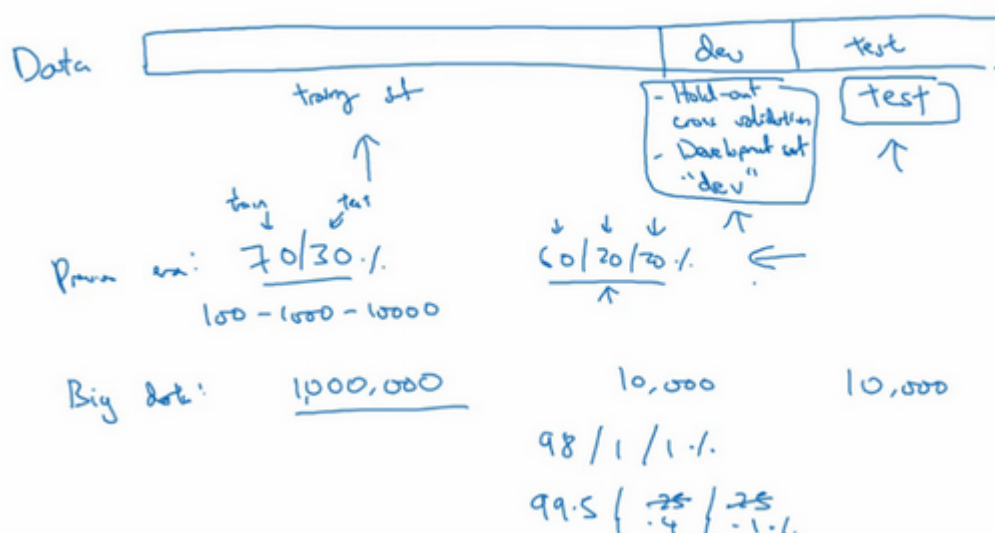
接下来，我们开始对训练执行算法，通过验证集或简单交叉验证集选择最好的模型，经过充分验证，我们选定了最终模型，然后就可以在测试集上进行评估了，为了无偏评估算法的运行状况。

在机器学习发展的小数据量时代，常见做法是将所有数据三七分，就是人们常说的70%验证集，30%测试集，如果没有明确设置验证集，也可以按照60%训练，20%验证和20%测试集来划分。这是前几年机器学习领域普遍认可的最好的实践方法。



如果只有100条，1000条或者1万条数据，那么上述比例划分是非常合理的。

但是在大数据时代，我们数据量可能是百万级别，那么验证集和测试集占数据总量的比例会趋向于变得更小。因为验证集的目的就是验证不同的算法，检验哪种算法更有效，因此，验证集要足够大才能评估，比如2个甚至10个不同算法，并迅速判断出哪种算法更有效。我们可能不需要拿出20%的数据作为验证集。

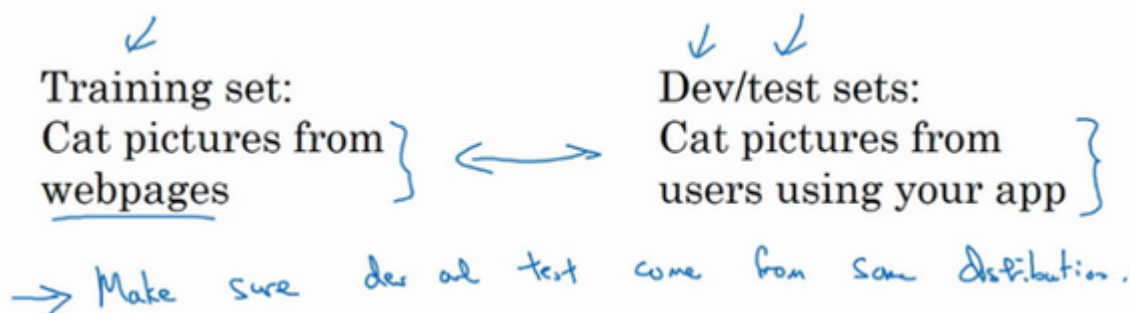


Andrew Ng

比如我们有100万条数据，那么取1万条数据便足以进行评估，找出其中表现最好的1-2种算法。同样地，根据最终选择的分类器，测试集的主要目的是正确评估分类器的性能，所以，如果拥有百万数据，我们只需要1000条数据，便足以评估单个分类器，并且准确评估该分类器的性能。假设我们有100万条数据，其中1万条作为验证集，1万条作为测试集，100万里取1万，比例是1%，即：训练集占98%，验证集和测试集各占1%。对于数据量过百万的应用，训练集可以占到99.5%，验证和测试集各占0.25%，或者验证集占0.4%，测试集占0.1%。

总结一下，在机器学习中，我们通常将样本分成训练集，验证集和测试集三部分，数据集规模相对较小，适用传统的划分比例，数据集规模较大的，验证集和测试集要小于数据总量的20%或10%。后面我会给出如何划分验证集和测试集的具体指导。

现代深度学习的另一个趋势是越来越多的人在训练和测试集分布不匹配的情况下进行训练，假设你要构建一个用户可以上传大量图片的应用程序，目的是找出并呈现所有猫咪图片，可能你的用户都是爱猫人士，训练集可能是从网上下载的猫咪图片，而验证集和测试集是用户在这个应用上上传的猫的图片，就是说，训练集可能是从网络上抓下来的图片。而验证集和测试集是用户上传的图片。结果许多网页上的猫咪图片分辨率很高，很专业，后期制作精良，而用户上传的照片可能是用手机随意拍摄的，像素低，比较模糊，这两类数据有所不同，针对这种情况，根据经验，我建议大家要确保验证集和测试集的数据来自同一分布，关于这个问题我也会多讲一些。因为你们要用验证集来评估不同的模型，尽可能地优化性能。如果验证集和测试集来自同一个分布就会很好。

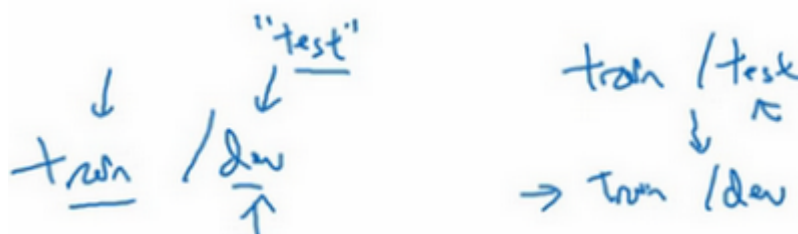


但由于深度学习算法需要大量的训练数据，为了获取更大规模的训练数据集，我们可以采用当前流行的各种创意策略，例如，网页抓取，代价就是训练集数据与验证集和测试集数据有可能不是来自同一分布。但只要遵循这个经验法则，你就会发现机器学习算法会变得更快。我会在后面的课程中更加详细地解释这条经验法则。

最后一点，就算没有测试集也不要紧，测试集的目的是对最终所选定的神经网络系统做出无偏估计，如果不需要无偏估计，也可以不设置测试集。所以如果只有验证集，没有测试集，我们要做的就是，在训练集上训练，尝试不同的模型框架，在验证集上评估这些模型，然后迭代并选出适用的模型。因为验证集中已经涵盖测试集数据，其不再提供无偏性能评估。当然，如果你不需要无偏估计，那就再好不过了。

Not having a test set might be okay. (Only dev set.)

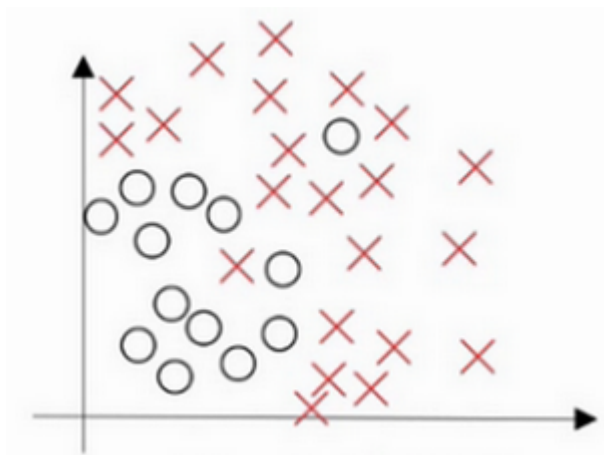
在机器学习中，如果只有一个训练集和一个验证集，而没有独立的测试集，遇到这种情况，训练集还被人们称为训练集，而验证集则被称为测试集，不过在实际应用中，人们只是把测试集当成简单交叉验证集使用，并没有完全实现该术语的功能，因为他们把验证集数据过度拟合到了测试集中。如果某团队跟你说他们只设置了一个训练集和一个测试集，我会很谨慎，心想他们是不是真的有训练验证集，因为他们把验证集数据过度拟合到了测试集中，让这些团队改变叫法，改称其为“训练验证集”，而不是“训练测试集”，可能不太容易。即便我认为“训练验证集”在专业用语上更准确。实际上，如果你不需要无偏评估算法性能，那么这样是可以的。



所以说，搭建训练验证集和测试集能够加速神经网络的集成，也可以更有效地衡量算法地偏差和方差，从而帮助我们更高效地选择合适方法来优化算法。

1.2 偏差，方差 (Bias /Variance)

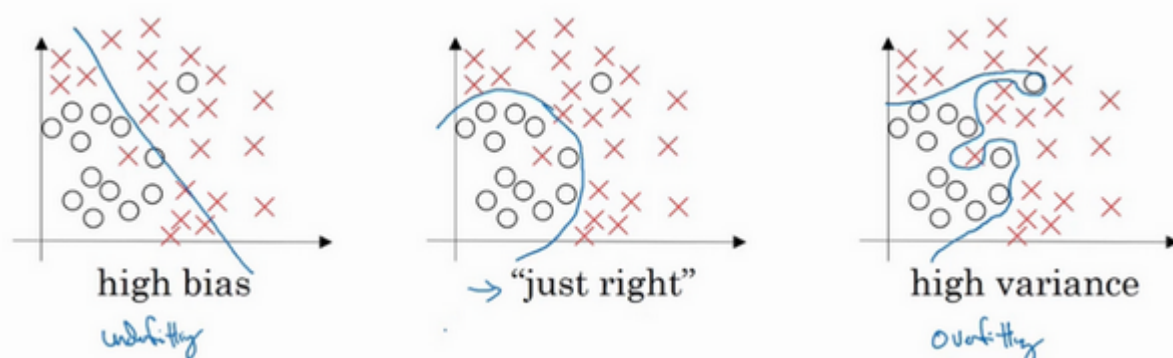
我注意到，几乎所有机器学习从业人员都期望深刻理解偏差和方差，这两个概念易学难精，即使你自己认为已经理解了偏差和方差的基本概念，却总有一些意想不到的新东西出现。关于深度学习的误差问题，另一个趋势是对偏差和方差的权衡研究甚浅，你可能听说过这两个概念，但深度学习的误差很少权衡二者，我们总是分别考虑偏差和方差，却很少谈及偏差和方差的权衡问题，下面我们来一探究竟。



假设这就是数据集，如果给这个数据集拟合一条直线，可能得到一个逻辑回归拟合，但它并不能很好地拟合该数据，这是高偏差（**high bias**）的情况，我们称为“欠拟合”（**underfitting**）。

相反的如果我们拟合一个非常复杂的分类器，比如深度神经网络或含有隐藏单元的神经网络，可能就非常适用于这个数据集，但是这看起来也不是一种很好的拟合方式分类器方差较高（**high variance**），数据过度拟合（**overfitting**）。

在两者之间，可能还有一些像图中这样的，复杂程度适中，数据拟合适度的分类器，这个数据拟合看起来更加合理，我们称之为“适度拟合”（**just right**）是介于过度拟合和欠拟合中间的一类。



在这样一个只有 x_1 和 x_2 两个特征的二维数据集中，我们可以绘制数据，将偏差和方差可视化。在多维空间数据中，绘制数据和可视化分割边界无法实现，但我们可以通过几个指标，来研究偏差和方差。

Bias and Variance

Cat classification



我们沿用猫咪图片分类这个例子，左边一张是猫咪图片，右边一张不是。理解偏差和方差的两个关键数据是训练集误差（**Train set error**）和验证集误差（**Dev set error**），为了方便论证，假设我们可以辨别图片中的小猫，我们用肉眼识别几乎是不会出错的。

假定训练集误差是1%，为了方便论证，假定验证集误差是11%，可以看出训练集设置得非常好，而验证集设置相对较差，我们可能过度拟合了训练集，在某种程度上，验证集并没有充分利用交叉验证集的作用，像这种情况，我们称之为“高方差”。

通过查看训练集误差和验证集误差，我们便可以诊断算法是否具有高方差。也就是说衡量训练集和验证集误差就可以得出不同结论。

假设训练集误差是15%，我们把训练集误差写在首行，验证集误差是16%，假设该案例中人的错误率几乎为0%，人们浏览这些图片，分辨出是不是猫。算法并没有在训练集中得到很好训练，如果训练数据的拟合度不高，就是数据欠拟合，可以说这种算法偏差比较高。相反，它对于验证集产生的结果却是合理的，验证集中的错误率只比训练集的多了1%，所以这种算法偏差高，因为它甚至不能拟合训练集，这与上一张幻灯片最左边的图片相似。

再举一个例子，训练集误差是15%，偏差相当高，但是，验证集的评估结果更糟糕，错误率达到30%，在这种情况下，我会认为这种算法偏差高，因为它在训练集上结果不理想，而且方差也很高，这是方差偏差都很糟糕的情况。

再看最后一个例子，训练集误差是0.5%，验证集误差是1%，用户看到这样的结果会很开心，猫咪分类器只有1%的错误率，偏差和方差都很低。

有一点我先在这个简单提一下，具体的留在后面课程里讲，这些分析都是基于假设预测的，假设人眼辨别的错误率接近0%，一般来说，最优误差也被称为贝叶斯误差，所以，最优误差接近0%，我就不在这里细讲了，如果最优误差或贝叶斯误差非常高，比如15%。我们再看看这个分类器（训练误差15%，验证误差16%），15%的错误率对训练集来说也是非常合理的，偏差不高，方差也非常低。

Train set error:	1%	15% ←	15%	0.5%
Dev set error:	11%	16% ←	30%	1%
	high variance	high bias ↑	high bias & high var	low bias low variance
Ham: 20%				
Optimal (Bayes) error: 2% to 15%				

当所有分类器都不适用时，如何分析偏差和方差呢？比如，图片很模糊，即使是人眼，或者没有系统可以准确无误地识别图片，在这种情况下，最优误差会更高，那么分析过程就要做些改变了，我们暂时先不讨论这些细微差别，重点是通过查看训练集误差，我们可以判断数据拟合情况，至少对于训练数据是这样，可以判断是否有偏差问题，然后查看错误率有多高。当完成训练集训练，开始使用验证集验证时，我们可以判断方差是否过高，从训练集到验证集的这个过程中，我们可以判断方差是否过高。

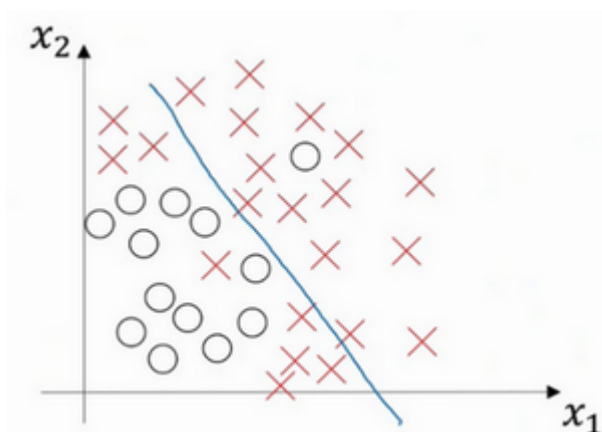
Train set error:	1%	15% ←	15%	0.5%
Dev set error:	11%	16% ←	30%	1%

以上分析的前提都是假设基本误差很小，训练集和验证集数据来自相同分布，如果没有这些假设作为前提，分析过程更加复杂，我们将会在稍后课程里讨论。

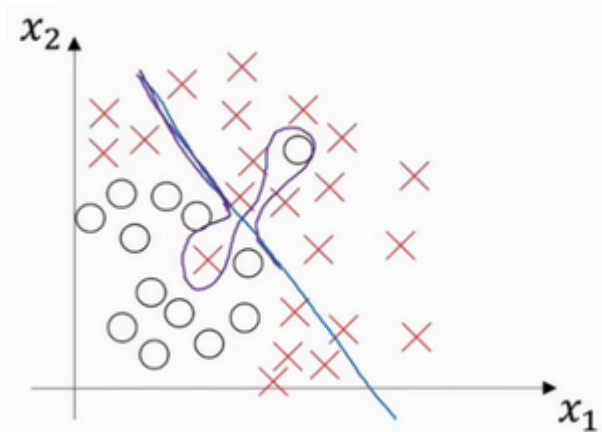
上一张幻灯片，我们讲了高偏差和高方差的情况，大家应该对优质分类器有了一定的认识，偏差和方差都高是什么样呢？这种情况对于两个衡量标准来说都是非常糟糕的。

Train set error:	1%	15% ←	15%	0.5%
Dev set error:	11%	16% ←	30%	1%
	high variance ↑	high bias ↑	high bias & high variance	low bias low variance ↑
<u>Ham: 20%</u>				

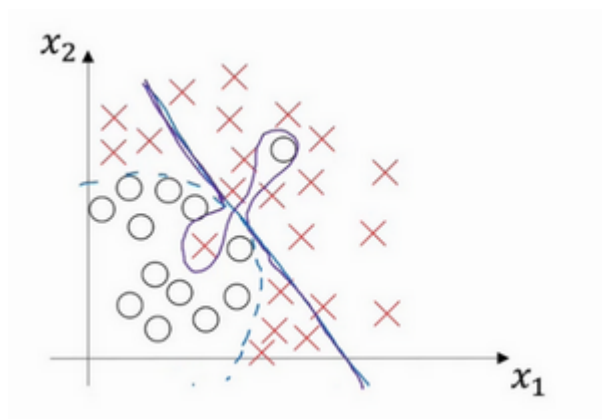
我们之前讲过，这样的分类器，会产生高偏差，因为它的数据拟合度低，像这种接近线性的分类器，数据拟合度低。



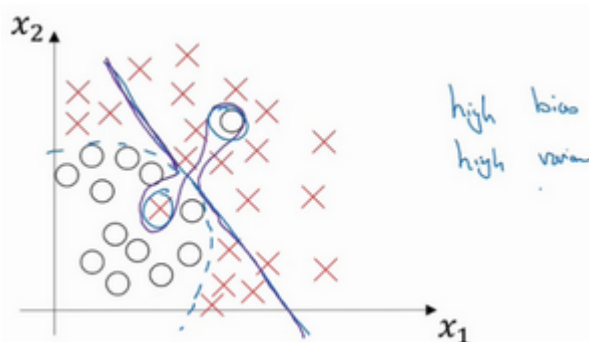
但是如果我们稍微改变一下分类器，我用紫色笔画出，它会过度拟合部分数据，用紫色线画出的分类器具有高偏差和高方差，偏差高是因为它几乎是一条线性分类器，并未拟合数据。



这种二次曲线能够很好地拟合数据。



这条曲线中间部分灵活性非常高，却过度拟合了这两个样本，这类分类器偏差很高，因为它几乎是线性的。



而采用曲线函数或二次元函数会产生高方差，因为它曲线灵活性太高以致拟合了这两个错误样本和中间这些活跃数据。

这看起来有些不自然，从两个维度上看都不太自然，但对于高维数据，有些数据区域偏差高，有些数据区域方差高，所以在高维数据中采用这种分类器看起来就不会那么牵强了。

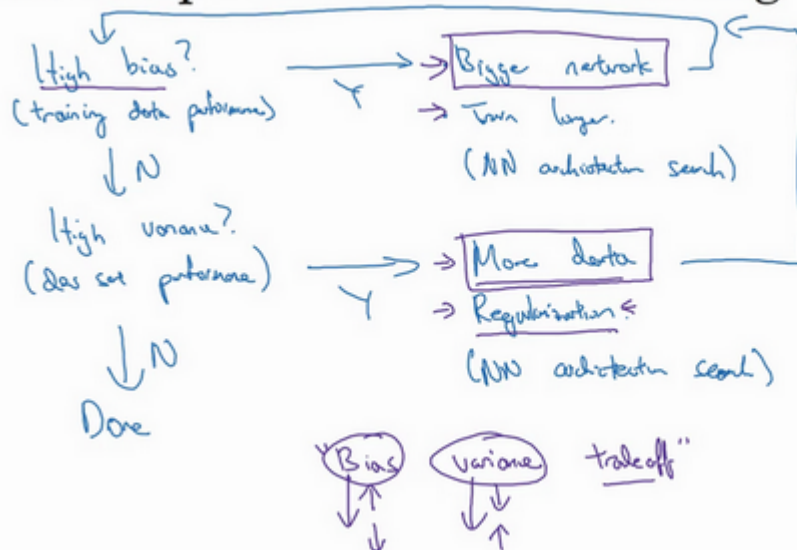
总结一下，我们讲了如何通过分析在训练集上训练算法产生的误差和验证集上验证算法产生的误差来诊断算法是否存在高偏差和高方差，是否两个值都高，或者两个值都不高，根据算法偏差和方差的具体情况决定接下来你要做的工作，下节课，我会根据算法偏差和方差的高低情况讲解一些机器学习的基本方法，帮助大家更系统地优化算法，我们下节课见。

1.3 机器学习基础（Basic Recipe for Machine Learning）

上节课我们讲的是如何通过训练误差和验证集误差判断算法偏差或方差是否偏高，帮助我们更加系统地在机器学习中运用这些方法来优化算法性能。

下图就是我在训练神经网络用到的基本方法：（尝试这些方法，可能有用，可能没用）

Basic recipe for machine learning



Andrew Ng

这是我在训练神经网络时用到地基本方法，初始模型训练完成后，我首先要知道算法的偏差高不高，如果偏差较高，试着评估训练集或训练数据的性能。如果偏差的确很高，甚至无法拟合训练集，那么你要做的就是选择一个新的网络，比如含有更多隐藏层或者隐藏单元的网络，或者花费更多时间来训练网络，或者尝试更先进的优化算法，后面我们会讲到这部分内容。你也可以尝试其他方法，可能有用，也可能没用。

一会儿我们会看到许多不同的神经网络架构，或许你能找到一个更合适解决此问题的新的网络架构，加上括号，因为其中一条就是你必须去尝试，可能有用，也可能没用，不过采用规模更大的网络通常都会有所帮助，延长训练时间不一定有用，但也没什么坏处。训练学习算法时，我会不断尝试这些方法，直到解决掉偏差问题，这是最低标准，反复尝试，直到可以拟合数据为止，至少能够拟合训练集。

如果网络足够大，通常可以很好的拟合训练集，只要你能扩大网络规模，如果图片很模糊，算法可能无法拟合该图片，但如果有人可以分辨出图片，如果你觉得基本误差不是很高，那么训练一个更大的网络，你就应该可以.....至少可以很好地拟合训练集，至少可以拟合或者过拟合训练集。一旦偏差降低到可以接受的数值，检查一下方差有没有问题，为了评估方差，我们要查看验证集性能，我们能从一个性能理想的训练集推断出验证集的性能是否也理想，如果方差高，最好的解决办法就是采用更多数据，如果你能做到，会有一定的帮助，但有时候，我们无法获得更多数据，我们也可以尝试通过正则化来减少过拟合，这个我们下节课会讲。有时候我们不得不反复尝试，但是，如果能找到更合适的神经网络框架，有时它可能会一箭双雕，同时减少方差和偏差。如何实现呢？想系统地说出做法很难，总之就是不断重复尝试，直到找到一个低偏差，低方差的框架，这时你就成功了。

有两点需要大家注意：

第一点，高偏差和高方差是两种不同的情况，我们后续要尝试的方法也可能完全不同，我通常会用训练验证集来诊断算法是否存在偏差或方差问题，然后根据结果选择尝试部分方法。举个例子，如果算法存在高偏差问题，准备更多训练数据其实也没什么用处，至少这不是更有效的方法，所以大家要清楚存在的问题是偏差还是方差，还是两者都有问题，明确这一点有助于我们选择出最有效的方法。

第二点，在机器学习的初期阶段，关于所谓的偏差方差权衡的讨论屡见不鲜，原因是我们能尝试的方法有很多。可以增加偏差，减少方差，也可以减少偏差，增加方差，但是在深度学习的早期阶段，我们没有太多工具可以做到只减少偏差或方差却不影响到另一方。但在当前的深度学习和大数据时代，只要持续训练一个更大的网络，只要准备了更多数据，那么也并非只有这两种情况，我们假定是这样，那么，只要正则适度，通常构建一个更大的网络便可以，在不影响方差的同时减少偏差，而采用更多数据通常可以在不过多影响偏差的同时减少方差。这两步实际要做的工作是：训练网络，选择网络或者准备更多数据，现在我们有工具可以做到在减少偏差或方差的同时，不对另一

方产生过多不良影响。我觉得这就是深度学习对监督式学习大有裨益的一个重要原因，也是我们不用太过关注如何平衡偏差和方差的一个重要原因，但有时我们有很多选择，减少偏差或方差而不增加另一方。最终，我们会得到一个非常规范化的网络。从下节课开始，我们将讲解正则化，训练一个更大的网络几乎没有任何负面影响，而训练一个大型神经网络的主要代价也只是计算时间，前提是网络是比较规范化的。

今天我们讲了如何通过组织机器学习来诊断偏差和方差的基本方法，然后选择解决问题的正确操作，希望大家有所了解和认识。我在课上不止一次提到了正则化，它是一种非常实用的减少方差的方法，正则化时会出现偏差方差权衡问题，偏差可能略有增加，如果网络足够大，增幅通常不会太高，我们下节课再细讲，以便大家更好理解如何实现神经网络的正则化。

1.4 正则化 (Regularization)

深度学习可能存在过拟合问题——高方差，有两个解决方法，一个是正则化，另一个是准备更多的数据，这是非常可靠的方法，但你可能无法时时刻刻准备足够多的训练数据或者获取更多数据的成本很高，但正则化通常有助于避免过拟合或减少你的网络误差。

如果你怀疑神经网络过度拟合了数据，即存在高方差问题，那么最先想到的方法可能是正则化，另一个解决高方差的方法就是准备更多数据，这也是非常可靠的办法，但你可能无法时时准备足够多的训练数据，或者，获取更多数据的成本很高，但正则化有助于避免过度拟合，或者减少网络误差，下面我们就来讲讲正则化的作用原理。

我们用逻辑回归来实现这些设想，求成本函数 J 的最小值，它是我们定义的成本函数，参数包含一些训练数据和不同数据中个体预测的损失， w 和 b 是逻辑回归的两个参数， w 是一个多维度参数矢量， b 是一个实数。在逻辑回归函数中加入正则化，只需添加参数 λ ，也就是正则化参数，一会儿再详细讲。

$\frac{\lambda}{2m}$ 乘以 w 范数的平方, w 欧几里德范数的平方等于 w_j (j 值从 1 到 n_x) 平方的和, 也可表示为 $w^T w$, 也就是向量参数 w 的欧几里德范数 (2 范数) 的平方, 此方法称为 $L2$ 正则化。因为这里用了欧几里德法线, 被称为向量参数 w 的 $L2$ 范数。

$$\min_{w,b} J(w,b)$$

为什么只正则化参数 w ？为什么不再加上参数 b 呢？你可以这么做，只是我习惯省略不写，因为 w 通常是一个高维参数矢量，已经可以表达高偏差问题， w 可能包含有很多参数，我们不可能拟合所有参数，而 b 只是单个数字，所以 w 几乎涵盖所有参数，而不是 b ，如果加了参数 b ，其实也没太大影响，因为 b 只是众多参数中的一个，所以我通常省略不计，如果你想加上这个参数，完全没问题。

$$J(\omega, b) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2m} \|\omega\|_2^2$$

$L2$ 正则化是最常见的正则化类型，你们可能听说过 $L1$ 正则化， $L1$ 正则化，加的不是 $L2$ 范数，而是正则项 $\frac{\lambda}{m}$ 乘以 $\sum_{j=1}^{n_x} |w_j|$ ， $\sum_{j=1}^{n_x} |w_j|$ 也被称为参数 w 向量的 $L1$ 范数，无论分母是 m 还是 $2m$ ，它都是一个比例常量。

$$L_1 \text{ regularization} \quad \frac{\lambda}{2m} \sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m} \|w\|_1 \quad w \text{ will be sparse}$$

如果用的是 L_1 正则化, w 最终会是稀疏的, 也就是说 w 向量中有很多0, 有人说这样有利于压缩模型, 因为集合中参数均为0, 存储模型所占用的内存更少。实际上, 虽然 L_1 正则化使模型变得稀疏, 却没有降低太多存储内存, 所以我认为这并不是 L_1 正则化的目的, 至少不是为了压缩模型, 人们在训练网络时, 越来越倾向于使用 L_2 正则化。

我们来看最后一个细节, λ 是正则化参数, 我们通常使用验证集或交叉验证集来配置这个参数, 尝试各种各样的数据, 寻找最好的参数, 我们要考虑训练集之间的权衡, 把参数设置为较小值, 这样可以避免过拟合, 所以 λ 是另外一个需要调整的超参数, 顺便说一下, 为了方便写代码, 在Python编程语言中, λ 是一个保留字段, 编写代码时, 我们删掉 λ , 写成 $lambda$, 以免与Python中的保留字段冲突, 这就是在逻辑回归函数中实现 L_2 正则化的过程, 如何在神经网络中实现 L_2 正则化呢?

Logistic regression

$$\min_{w,b} J(w,b) \quad w \in \mathbb{R}^{n_x}, b \in \mathbb{R} \quad \lambda = \text{regularization parameter}$$

$$J(w,b) = \frac{1}{m} \sum_{i=1}^m \ell(y^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

L_2 regularization $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w \leftarrow$

L_1 regularization $\frac{\lambda}{2m} \sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m} \|w\|_1 \quad w \text{ will be sparse}$

~~$\frac{\lambda}{2m} b^2$~~ omit

神经网络含有一个成本函数, 该函数包含 $W^{[1]}, b^{[1]}$ 到 $W^{[L]}, b^{[L]}$ 所有参数, 字母 L 是神经网络所含的层数, 因此成本函数等于 m 个训练样本损失函数的总和乘以 $\frac{1}{m}$, 正则项为 $\frac{\lambda}{2m} \sum_{l=1}^L \|W^{[l]}\|^2$, 我们称 $\|W^{[l]}\|^2$ 为范数平方, 这个矩阵范数 $\|W^{[l]}\|^2$ (即平方范数), 被定义为矩阵中所有元素的平方求和,

Neural network

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m \ell(y^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2$$

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2$$

"Frobenius norm" $\|\cdot\|_2^2 \quad \|\cdot\|_F^2$

$w: \begin{pmatrix} n^{[l-1]} & n^{[l]} \end{pmatrix}$

我们看下求和公式的具体参数, 第一个求和符号其值 i 从1到 $n^{[l-1]}$, 第二个其 j 值从1到 $n^{[l]}$, 因为 W 是一个 $n^{[l]} \times n^{[l-1]}$ 的多维矩阵, $n^{[l]}$ 表示 l 层单元的数量, $n^{[l-1]}$ 表示第 $l-1$ 层隐藏单元的数量。

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2$$

$w^{[l]}: \begin{pmatrix} n^{[l-1]} & n^{[l]} \end{pmatrix}$

该矩阵范数被称作“弗罗贝尼乌斯范数”，用下标 F 标注，鉴于线性代数中一些神秘晦涩的原因，我们不称之为“矩阵 $L2$ 范数”，而称它为“弗罗贝尼乌斯范数”，矩阵 $L2$ 范数听起来更自然，但鉴于一些大家无须知道的特殊原因，按照惯例，我们称之为“弗罗贝尼乌斯范数”，它表示一个矩阵中所有元素的平方和。

"Frobenius norm"
 $\| \cdot \|_2^2$
 $\| \cdot \|_F^2$

该如何使用该范数实现梯度下降呢？

用**backprop**计算出 dW 的值，**backprop**会给出 J 对 W 的偏导数，实际上是 $W^{[l]}$ ，把 $W^{[l]}$ 替换为 $W^{[l]}$ 减去学习率乘以 dW 。

$dW^{[l]} = (\text{from backprop})$
 $\frac{\partial J}{\partial W^{[l]}}$
 $W^{[l]} := W^{[l]} - \alpha dW^{[l]}$

这就是之前我们额外增加的正则化项，既然已经增加了这个正则项，现在我们要做的就是给 dW 加上这一项 $\frac{\lambda}{m} W^{[l]}$ ，然后计算这个更新项，使用新定义的 $dW^{[l]}$ ，它的定义含有相关参数代价函数导数和，以及最后添加的额外正则项，这也是 $L2$ 正则化有时被称为“权重衰减”的原因。

$dW^{[l]} = (\text{from backprop}) + \frac{\lambda}{m} W^{[l]}$
 $\frac{\partial J}{\partial W^{[l]}} = dW^{[l]}$
 $\rightarrow W^{[l]} := W^{[l]} - \alpha dW^{[l]}$
 "Weight decay"

我们用 $dW^{[l]}$ 的定义替换此处的 $dW^{[l]}$ ，可以看到， $W^{[l]}$ 的定义被更新为 $W^{[l]}$ 减去学习率 α 乘以**backprop**再加上 $\frac{\lambda}{m} W^{[l]}$ 。

$W^{[l]} := W^{[l]} - \alpha \left[(\text{from backprop}) + \frac{\lambda}{m} W^{[l]} \right]$
 $= W^{[l]} - \frac{\alpha \lambda}{m} W^{[l]} - \alpha (\text{from backprop})$

△

该正则项说明，不论 $W^{[l]}$ 是什么，我们都试图让它变得更小，实际上，相当于我们给矩阵 W 乘以 $(1 - \alpha \frac{\lambda}{m})$ 倍的权重，矩阵 W 减去 $\alpha \frac{\lambda}{m}$ 倍的它，也就是用这个系数 $(1 - \alpha \frac{\lambda}{m})$ 乘以矩阵 W ，该系数小于1，因此 $L2$ 范数正则化也被称为“权重衰减”，因为它就像一般的梯度下降， W 被更新为少了 α 乘以**backprop**输出的最初梯度值，同时 W 也乘以了这个系数，这个系数小于1，因此 $L2$ 正则化也被称为“权重衰减”。

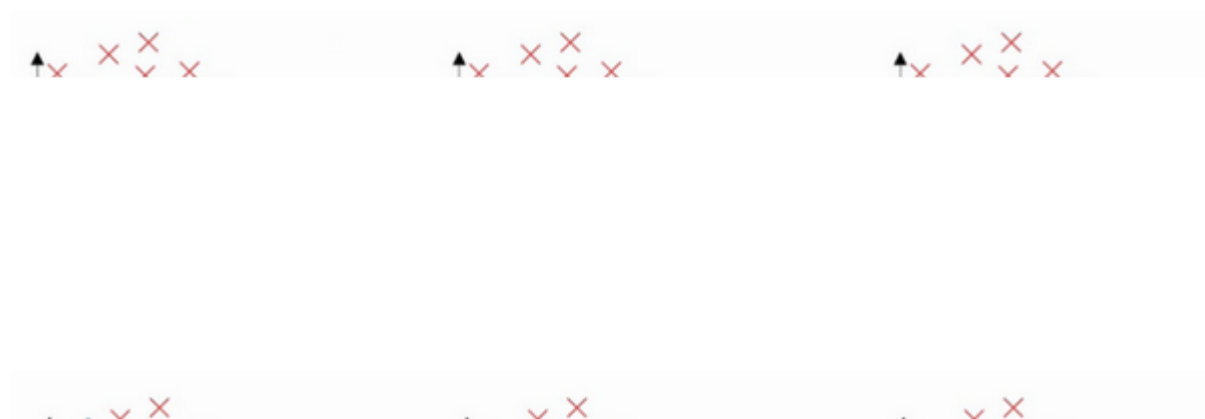
$$w_2 := w_2 - \frac{\partial L}{\partial w_2} + \lambda w_2$$

我不打算这么叫它，之所以叫它“权重衰减”是因为这两项相等，权重指标乘以一个小于1的系数。

以上就是在神经网络中应用L2正则化的过程，有人会问我，为什么正则化可以预防过拟合，我们放在下节课讲，同时直观感受一下正则化是如何预防过拟合的。

1.5 为什么正则化有利于预防过拟合呢？（Why regularization reduces overfitting?）

为什么正则化有利于预防过拟合呢？为什么它可以减少方差问题？我们通过两个例子来直观体会一下。



左图是高偏差，右图是高方差，中间是**Just Right**，这几张图我们在前面课程中看到过。

现在我们来看下这个庞大的深度拟合神经网络。我知道这张图不够大，深度也不够，但你可以想象这是一个过拟合的神经网络。这是我们的代价函数 J ，含有参数 W ， b 。我们添加正则项，它可以避免数据权值矩阵过大，这就是弗罗贝尼乌斯范数，为什么压缩L2范数，或者弗罗贝尼乌斯范数或者参数可以减少过拟合？

直观上理解就是如果正则化 λ 设置得足够大，权重矩阵 W 被设置为接近于0的值，直观理解就是把多隐藏单元的权重设为0，于是基本上消除了这些隐藏单元的许多影响。如果是这种情况，这个被大大简化了的神经网络会变成一个很小的网络，小到如同一个逻辑回归单元，可是深度却很大，它会使这个网络从过度拟合的状态更接近左图的高偏差状态。

但是 λ 会存在一个中间值，于是会有一个接近“Just Right”的中间状态。

直观理解就是 λ 增加到足够大， W 会接近于0，实际上是不会发生这种情况的，我们尝试消除或至少减少许多隐藏单元的影响，最终这个网络会变得更简单，这个神经网络越来越接近逻辑回归，我们直觉上认为大量隐藏单元被完全消除了，其实不然，实际上是该神经网络的所有隐藏单元依然存在，但是它们的影响变得更小了。神经网络变得更简单了，貌似这样更不容易发生过拟合，因此我不确定这个直觉经验是否有用，不过在编程中执行正则化时，你实际看到一些方差减少的结果。

我们再来直观感受一下，正则化为什么可以预防过拟合，假设我们用的是这样的双曲线激活函数。

用 $g(z)$ 表示 $\tanh(z)$ ，那么我们发现，只要 z 非常小，如果 z 只涉及少量参数，这里我们利用了双曲正切函数的线性状态，只要 z 可以扩展为这样的更大值或者更小值，激活函数开始变得非线性。

现在你应该摒弃这个直觉，如果正则化参数 λ 很大，激活函数的参数会相对较小，因为代价函数中的参数变大了，如果 W 很小，

如果 W 很小，相对来说， z 也会很小。

特别是，如果 z 的值最终在这个范围内，都是相对较小的值， $g(z)$ 大致呈线性，每层几乎都是线性的，和线性回归函数一样。

第一节课我们讲过，如果每层都是线性的，那么整个网络就是一个线性网络，即使是一个非常深的深层网络，因具有线性激活函数的特征，最终我们只能计算线性函数，因此，它不适用于非常复杂的决策，以及过度拟合数据集的非线性决策边界，如同我们在幻灯片中看到的过度拟合高方差的情况。

总结一下，如果正则化参数变得很大，参数 W 很小， z 也会相对变小，此时忽略 b 的影响， z 会相对变小，实际上， z 的取值范围很小，这个激活函数，也就是曲线函数 $\tanh$ 会相对呈线性，整个神经网络会计算离线性函数近的值，这个线性函数非常简单，并不是一个极复杂的高度非线性函数，不会发生过拟合。

大家在编程作业里实现正则化的时候，会亲眼看到这些结果，总结正则化之前，我给大家一个执行方面的小建议，在增加正则化项时，应用之前定义的代价函数 J ，我们做过修改，增加了一项，目的是预防权重过大。

如果你使用的是梯度下降函数，在调试梯度下降时，其中一步就是把代价函数 J 设计成这样一个函数，在调试梯度下降时，它代表梯度下降的调幅数量。可以看到，代价函数对于梯度下降的每个调幅都单调递减。如果你实施的是正则化函数，请牢记， J 已经有一个全新的定义。如果你用的是原函数 J ，也就是这第一个项正则化项，你可能看不到单调递减现象，为了调试梯度下降，请务必使用新定义的 J 函数，它包含第二个正则化项，否则函数 J 可能不会在所有调幅范围内都单调递减。

这就是 $L2$ 正则化，它是我在训练深度学习模型时最常用的一种方法。在深度学习中，还有一种方法也用到了正则化，就是dropout正则化，我们下节课再讲。

1.6 dropout 正则化 (Dropout Regularization)

除了 $L2$ 正则化，还有一个非常实用的正则化方法——“Dropout (随机失活)”，我们来看看它的工作原理。

假设你在训练上图这样的神经网络，它存在过拟合，这就是**dropout**所要处理的，我们复制这个神经网络，**dropout**会遍历网络的每一层，并设置消除神经网络中节点的概率。假设网络中的每一层，每个节点都以抛硬币的方式设置概率，每个节点得以保留和消除的概率都是0.5，设置完节点概率，我们会消除一些节点，然后删除掉从该节点进出的连线，最后得到一个节点更少，规模更小的网络，然后用**backprop**方法进行训练。

这是网络节点精简后的一个样本，对于其它样本，我们照旧以抛硬币的方式设置概率，保留一类节点集合，删除其它类型的节点集合。对于每个训练样本，我们都将采用一个精简后神经网络来训练它，这种方法似乎有点怪，单纯遍历节点，编码也是随机的，可它真的有效。不过可想而知，我们针对每个训练样本训练规模极小的网络，最后你可能会认识到为什么要正则化网络，因为我们在训练极小的网络。

如何实施**dropout**呢？方法有几种，接下来我要讲的是最常用的方法，即**inverted dropout**（反向随机失活），出于完整性考虑，我们用一个三层（ $l = 3$ ）网络来举例说明。编码中会有很多涉及到3的地方。我只举例说明如何在某一层中实施**dropout**。

首先要定义向量 d ， $d^{[3]}$ 表示一个三层的**dropout**向量：

```
d3 = np.random.rand(a3.shape[0],a3.shape[1])
```

然后看它是否小于某数，我们称之为**keep-prob**，**keep-prob**是一个具体数字，上个示例中它是0.5，而本例中它是0.8，它表示保留某个隐藏单元的概率，此处**keep-prob**等于0.8，它意味着消除任意一个隐藏单元的概率是0.2，它的作用就是生成随机矩阵，如果对 $a^{[3]}$ 进行因子分解，效果也是一样的。 $d^{[3]}$ 是一个矩阵，每个样本和每个隐藏单元，其中 $d^{[3]}$ 中的对应值为1的概率都是0.8，对应为0的概率是0.2，随机数字小于0.8。它等于1的概率是0.8，等于0的概率是0.2。

接下来要做的就是从第三层中获取激活函数，这里我们叫它 $a^{[3]}$ ， $a^{[3]}$ 含有要计算的激活函数， $a^{[3]}$ 等于上面的 $a^{[3]}$ 乘以 $d^{[3]}$ ，`a3=np.multiply(a3,d3)`，这里是元素相乘，也可写为 $a3 * d3$ ，它的作用就是让 $d^{[3]}$ 中所有等于0的元素（输出），而各个元素等于0的概率只有20%，乘法运算最终把 $d^{[3]}$ 中相应元素输出，即让 $d^{[3]}$ 中0元素与 $a^{[3]}$ 中相对元素归零。

如果用**python**实现该算法的话， $d^{[3]}$ 则是一个布尔型数组，值为**true**和**false**，而不是1和0，乘法运算依然有效，**python**会把**true**和**false**翻译为1和0，大家可以用**python**尝试一下。

最后，我们向外扩展 $a^{[3]}$ ，用它除以0.8，或者除以**keep-prob**参数。

下面我解释一下为什么要这么做，为方便起见，我们假设第三隐藏层上有50个单元或50个神经元，在一维上 $a^{[3]}$ 是50，我们通过因子分解将它拆分成 $50 \times m$ 维的，保留和删除它们的概率分别为80%和20%，这意味着最后被删除或归零的单元平均有10（ $50 \times 20\% = 10$ ）个，现在看下 $z^{[4]}$ ， $z^{[4]} = w^{[4]} a^{[3]} + b^{[4]}$ ，我们的预期是， $a^{[3]}$ 减少20%，也就是说 $a^{[3]}$ 中有20%的元素被归零，为了不影响 $z^{[4]}$ 的期望值，我们需要用 $w^{[4]} a^{[3]} / 0.8$ ，它将会修正或弥补我们所需的那20%， $a^{[3]}$ 的期望值不会变，划线部分就是所谓的**dropout**方法。

它的功能是，不论**keep-prop**的值是多少0.8，0.9甚至是1，如果**keep-prop**设置为1，那么就不存在**dropout**，因为它会保留所有节点。反向随机失活（**inverted dropout**）方法通过除以**keep-prob**，确保 $a^{[3]}$ 的期望值不变。

事实证明，在测试阶段，当我们评估一个神经网络时，也就是用绿线框标注的反向随机失活方法，使测试阶段变得更容易，因为它的扩展问题变少，我们将在下节课讨论。

据我了解，目前实施**dropout**最常用的方法就是**Inverted dropout**，建议大家动手实践一下。**Dropout**早期的迭代版本都没有除以**keep-prob**，所以在测试阶段，平均值会变得越来越复杂，不过那些版本已经不再使用了。

现在你使用的是 d 向量，你会发现，不同的训练样本，清除不同的隐藏单元也不同。实际上，如果你通过相同训练集多次传递数据，每次训练数据的梯度不同，则随机对不同隐藏单元归零，有时却并非如此。比如，需要将相同隐藏单元归零，第一次迭代梯度下降时，把一些隐藏单元归零，第二次迭代梯度下降时，也就是第二次遍历训练集时，对不同类型的隐藏层单元归零。向量 d 或 $d^{[3]}$ 用来决定第三层中哪些单元归零，无论用**foreprop**还是**backprop**，这里我们只介绍了**foreprop**。

如何在测试阶段训练算法，在测试阶段，我们已经给出了 x ，或是想预测的变量，用的是标准计数法。我用 $a^{[0]}$ ，第0层的激活函数标注为测试样本 x ，我们在测试阶段不使用**dropout**函数，尤其是像下列情况：

$$z^{[1]} = w^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = \dots$$

以此类推直到最后一层，预测值为 $\hat{y}$ 。

显然在测试阶段，我们并未使用**dropout**，自然也就不需要抛硬币来决定失活概率，以及要消除哪些隐藏单元了，因为在测试阶段进行预测时，我们不期望输出结果是随机的，如果测试阶段应用**dropout**函数，预测会受到干扰。理论上，你只需要多次运行预测处理过程，每一次，不同的隐藏单元会被随机归零，预测处理遍历它们，但计算效率低，得出的结果也几乎相同，与这个不同程序产生的结果极为相似。

Inverted dropout函数在除以**keep-prob**时可以记住上一步的操作，目的是确保即使在测试阶段不执行**dropout**来调整数值范围，激活函数的预期结果也不会发生变化，所以没必要在测试阶段额外添加尺度参数，这与训练阶段不同。

$$l = keep - prob$$

这就是**dropout**，大家可以通过本周的编程练习来执行这个函数，亲身实践一下。

为什么**dropout**会起作用呢？下节课我们将更加直观地了解**dropout**的具体功能。

1.7 理解 dropout (Understanding Dropout)

Dropout可以随机删除网络中的神经单元，他为什么可以通过正则化发挥如此大的作用呢？

直观上理解：不要依赖于任何一个特征，因为该单元的输入可能随时被清除，因此该单元通过这种方式传播下去，并为单元的四个输入增加一点权重，通过传播所有权重，**dropout**将产生收缩权重的平方范数的效果，和之前讲的 $L2$ 正则化类似；实施**dropout**的结果实它会压缩权重，并完成一些预防过拟合的外层正则化； $L2$ 对不同权重的衰减是不同的，它取决于激活函数倍增的大小。

总结一下，**dropout**的功能类似于 $L2$ 正则化，与 $L2$ 正则化不同的是应用方式不同会带来一点点小变化，甚至更适用于不同的输入范围。

第二个直观认识是，我们从单个神经元入手，如图，这个单元的工作就是输入并生成一些有意义的输出。通过**dropout**，该单元的输入几乎被消除，有时这两个单元会被删除，有时会删除其它单元，就是说，我用紫色圈起来的这个单元，它不能依靠任何特征，因为特征都有可能被随机清除，或者说该单元的输入也都有可能被随机清除。我不愿意把所有赌注都放在一个节点上，不愿意给任何一个输入加上太多权重，因为它可能会被删除，因此该单元将通过这种方式积极地传播开，并为单元的四个输入增加一点权重，通过传播所有权重，**dropout**将产生收缩权重的平方范数的效果，和我们之前讲过的 $L2$ 正则化类似，实施**dropout**的结果是它会压缩权重，并完成一些预防过拟合的外层正则化。

事实证明，**dropout**被正式地作为一种正则化的替代形式， L_2 对不同权重的衰减是不同的，它取决于倍增的激活函数的大小。

总结一下，**dropout**的功能类似于 L_2 正则化，与 L_2 正则化不同的是，被应用的方式不同，**dropout**也会有所不同，甚至更适用于不同的输入范围。

实施**dropout**的另一个细节是，这是一个拥有三个输入特征的网络，其中一个要选择的参数是**keep-prob**，它代表每一层上保留单元的概率。所以不同层的**keep-prob**也可以变化。第一层，矩阵 $W^{[1]}$ 是 7×3 ，第二个权重矩阵 $W^{[2]}$ 是 7×7 ，第三个权重矩阵 $W^{[3]}$ 是 3×7 ，以此类推， $W^{[2]}$ 是最大的权重矩阵，因为 $W^{[2]}$ 拥有最大参数集，即 7×7 ，为了预防矩阵的过拟合，对于这一层，我认为这是第二层，它的**keep-prob**值应该相对较低，假设是0.5。对于其它层，过拟合的程度可能没那么严重，它们的**keep-prob**值可能高一些，可能是0.7，这里是0.7。如果在某一层，我们不必担心其过拟合的问题，那么**keep-prob**可以为1，为了表达清除，我用紫色线笔把它们圈出来，每层**keep-prob**的值可能不同。

注意**keep-prob**的值是1，意味着保留所有单元，并且不在这一层使用**dropout**，对于有可能出现过拟合，且含有诸多参数的层，我们可以把**keep-prob**设置成比较小的值，以便应用更强大的**dropout**，有点像在处理 L_2 正则化的正则化参数 λ ，我们尝试对某些层施行更多正则化，从技术上讲，我们也可以对输入层应用**dropout**，我们有机会删除一个或多个输入特征，虽然现实中我们通常不这么做，**keep-prob**的值为1，是非常常用的输入值，也可以用更大的值，或许是0.9。但是消除一半的输入特征是不太可能的，如果我们遵守这个准则，**keep-prob**会接近于1，即使你对输入层应用**dropout**。

总结一下，如果你担心某些层比其它层更容易发生过拟合，可以把某些层的**keep-prob**值设置得比其它层更低，缺点是为了使用交叉验证，你要搜索更多的超参数，另一种方案是在一些层上应用**dropout**，而有些层不用**dropout**，应用**dropout**的层只含有一个超参数，就是**keep-prob**。

结束前分享两个实施过程中的技巧，实施**dropout**，在计算机视觉领域有很多成功的第一次。计算视觉中的输入量非常大，输入太多像素，以至于没有足够的参数，所以**dropout**在计算机视觉中应用得比较频繁，有些计算机视觉研究人员非常喜欢用它，几乎成了默认的选择，但要牢记一点，**dropout**是一种正则化方法，它有助于预防过拟合，因此除非算法过拟合，不然我是不会使用**dropout**的，所以它在其它领域应用得比较少，主要存在于计算机视觉领域，因为我们通常没有足够的参数，所以一直存在过拟合，这就是有些计算机视觉研究人员如此钟情于**dropout**函数的原因。直观上我认为不能概括其它学科。

dropout一大缺点就是代价函数 J 不再被明确定义，每次迭代，都会随机移除一些节点，如果再三检查梯度下降的性能，实际上是很难进行复查的。定义明确的代价函数 J 每次迭代后都会下降，因为我们所优化的代价函数 J 实际上并没有明确定义，或者说在某种程度上很难计算，所以我们失去了调试工具来绘制这样的图片。我通常会关闭**dropout**函数，将**keep-prob**的值设为1，运行代码，确保函数单调递减。然后打开**dropout**函数，希望在**dropout**过程中，代码并未引入bug。我觉得你也可以尝试其它方法，虽然我们并没有关于这些方法性能的数据统计，但你可以把它们与**dropout**方法一起使用。

1.8 其他正则化方法（Other regularization methods）

除了 L_2 正则化和随机失活（**dropout**）正则化，还有几种方法可以减少神经网络中的过拟合：

一.数据扩增

假设你正在拟合猫咪图片分类器，如果你想通过扩增训练数据来解决过拟合，但扩增数据代价高，而且有时候我们无法扩增数据，但我们可以通过添加这类图片来增加训练集。例如，水平翻转图片，并把它添加到训练集。所以现在训练集中有原图，还有翻转后的这张图片，所以通过水平翻转图片，训练集则可以增大一倍，因为训练集有冗余，这虽然不如我们额外收集一组新图片那么好，但这样做节省了获取更多猫咪图片的花费。

除了水平翻转图片，你也可以随意裁剪图片，这张图是把原图旋转并随意放大后裁剪的，仍能辨别出图片中的猫咪。

通过随意翻转和裁剪图片，我们可以增大数据集，额外生成假训练数据。和全新的，独立的猫咪图片数据相比，这些额外的假的数据无法包含像全新数据那么多的信息，但我们这么做基本没有花费，代价几乎为零，除了一些对抗性代价。以这种方式扩增算法数据，进而正则化数据集，减少过拟合比较廉价。

像这样人工合成数据的话，我们要通过算法验证，图片中的猫经过水平翻转之后依然是猫。大家注意，我并没有垂直翻转，因为我们不想上下颠倒图片，也可以随机选取放大后的部分图片，猫可能还在上面。

对于光学字符识别，我们还可以通过添加数字，随意旋转或扭曲数字来扩增数据，把这些数字添加到训练集，它们仍然是数字。为了方便说明，我对字符做了强变形处理，所以数字4看起来是波形的，其实不用对数字4做这么夸张的扭曲，只要轻微的变形就好，我做成这样是为了让大家看的更清楚。实际操作的时候，我们通常对字符做更轻微的变形处理。因为这几个4看起来有点扭曲。所以，数据扩增可作为正则化方法使用，实际功能上也与正则化相似。

二.early stopping

还有另外一种常用的方法叫作**early stopping**，运行梯度下降时，我们可以绘制训练误差，或只绘制代价函数 J 的优化过程，在训练集上用0-1记录分类误差次数。呈单调下降趋势，如图。

因为在训练过程中，我们希望训练误差，代价函数 J 都在下降，通过**early stopping**，我们不但可以绘制上面这些内容，还可以绘制验证集误差，它可以是验证集上的分类误差，或验证集上的代价函数，逻辑损失和对数损失等，你会发现，验证集误差通常会先呈下降趋势，然后在某个节点处开始上升，**early stopping**的作用是，你会说，神经网络已经在这个迭代过程中表现得很好了，我们在此停止训练吧，得到验证集误差，它是怎么发挥作用的？

当你还未在神经网络上运行太多迭代过程的时候，参数 w 接近0，因为随机初始化 w 值时，它的值可能都是较小的随机值，所以在你长期训练神经网络之前 w 依然很小，在迭代过程和训练过程中 w 的值会变得越来越来大，比如在这儿，神经网络中参数 w 的值已经非常大了，所以**early stopping**要做就是在中间点停止迭代过程，我们得到一个 w 值中等大小的弗罗贝尼乌斯范数，与 $L2$ 正则化相似，选择参数 w 范数较小的神经网络，但愿你的神经网络过度拟合不严重。

术语**early stopping**代表提早停止训练神经网络，训练神经网络时，我有时会用到**early stopping**，但是它也有一个缺点，我们来了解一下。

我认为机器学习过程包括几个步骤，其中一步是选择一个算法来优化代价函数 J ，我们有很多种工具来解决这个问题，如梯度下降，后面我会介绍其它算法，例如**Momentum**，**RMSprop**和**Adam**等等，但是优化代价函数 J 之后，我也不想发生过拟合，也有一些工具可以解决该问题，比如正则化，扩增数据等等。

在机器学习中，超级参数激增，选出可行的算法也变得越来越复杂。我发现，如果我们用一组工具优化代价函数 J ，机器学习就会变得更简单，在重点优化代价函数 J 时，你只需要留意 w 和 b ， $J(w, b)$ 的值越小越好，你只需要想办法减小这个值，其它的不用关注。然后，预防过拟合还有其他任务，换句话说就是减少方差，这一步我们用另外一套工具来实现，这个原理有时被称为“正交化”。思路就是在一个时间做一个任务，后面课上我会具体介绍正交化，如果你还不了解这个概念，不用担心。

但对我来说**early stopping**的主要缺点就是你不能独立地处理这两个问题，因为提早停止梯度下降，也就是停止了优化代价函数 J ，因为现在你不再尝试降低代价函数 J ，所以代价函数 J 的值可能不够小，同时你又希望不出现过拟合，你没有采取不同的方式来解决这两个问题，而是用一种方法同时解决两个问题，这样做的结果是我要考虑的东西变得更复杂。

如果不用**early stopping**，另一种方法就是 $L2$ 正则化，训练神经网络的时间就可能很长。我发现，这导致超级参数搜索空间更容易分解，也更容易搜索，但是缺点在于，你必须尝试很多正则化参数 λ 的值，这也导致搜索大量 λ 值的计算代价太高。

Early stopping的优点是，只运行一次梯度下降，你可以找出 w 的较小值，中间值和较大值，而无需尝试 $L2$ 正则化超级参数 λ 的很多值。

如果你还不能完全理解这个概念，没关系，下节课我们会详细讲解正交化，这样会更好理解。

虽然 $L2$ 正则化有缺点，可还是有很多人愿意用它。吴恩达老师个人更倾向于使用 $L2$ 正则化，尝试许多不同的 λ 值，假设你可以负担大量计算的代价。而使用**early stopping**也能得到相似结果，还不用尝试这么多 λ 值。

这节课我们讲了如何使用数据扩增，以及如何使用**early stopping**降低神经网络中的方差或预防过拟合。

1.9 归一化输入 (Normalizing inputs)

训练神经网络，其中一个加速训练的方法就是归一化输入。假设一个训练集有两个特征，输入特征为2维，归一化需要两个步骤：

1. 零均值
2. 归一化方差；

我们希望无论是训练集和测试集都是通过相同的 μ 和 σ^2 定义的数据转换，这两个是由训练集得出来的。

第一步是零均值化， $\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$ ，它是一个向量， x 等于每个训练数据 x 减去 μ ，意思是移动训练集，直到它完成零均值化。

第二步是归一化方差，注意特征 x_1 的方差比特征 x_2 的方差要大得多，我们要做的是给 σ 赋值， $\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)})^2$ ，这是节点 y 的平方， σ^2 是一个向量，它的每个特征都有方差，注意，我们已经完成零值均化， $(x^{(i)})^2$ 元素 y^2 就是方差，我们把所有数据除以向量 σ^2 ，最后变成上图形式。

x_1 和 x_2 的方差都等于1。提示一下，如果你用它来调整训练数据，那么用相同的 μ 和 σ^2 来归一化测试集。尤其是，你不希望训练集和测试集的归一化有所不同，不论 μ 的值是什么，也不论 σ^2 的值是什么，这两个公式中都会用到它们。所以你要用同样的方法调整测试集，而不是在训练集和测试集上分别预估 μ 和 σ^2 。因为我们希望不论是训练数据还是测试数据，都是通过相同 μ 和 σ^2 定义的相同数据转换，其中 μ 和 σ^2 是由训练集数据计算得来的。

我们为什么要这么做呢？为什么我们想要归一化输入特征，回想一下右上角所定义的代价函数。

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

如果你使用非归一化的输入特征，代价函数会像这样：

这是一个非常细长狭窄的代价函数，你要找的最小值应该在这里。但如果特征值在不同范围，假如 x_1 取值范围从1到1000，特征 x_2 的取值范围从0到1，结果是参数 w_1 和 w_2 值的范围或比率将会非常不同，这些数据轴应该是 w_1 和 w_2 ，但直观理解，我标记为 w 和 b ，代价函数就有点像狭长的碗一样，如果你能画出该函数的部分轮廓，它会是这样一个狭长的函数。

然而如果你归一化特征，代价函数平均起来看更对称，如果你在上图这样的代价函数上运行梯度下降法，你必须使用一个非常小的学习率。因为如果是在这个位置，梯度下降法可能需要多次迭代过程，直到最后找到最小值。但如果函数是一个更圆的球形轮廓，那么不论从哪个位置开始，梯度下降法都能够更直接地找到最小值，你可以在梯度下降法中使用较大步长，而不需要像在左图中那样反复执行。

当然，实际上 w 是一个高维向量，因此用二维绘制 w 并不能正确地传达并直观理解，但总地直观理解是代价函数会更圆一些，而且更容易优化，前提是特征都在相似范围内，而不是从1到1000，0到1的范围，而是在-1到1范围内或相似偏差，这使得代价函数 J 优化起来更简单快速。

实际上如果假设特征 x_1 范围在0-1之间， x_2 的范围在-1到1之间， x_3 范围在1-2之间，它们是相似范围，所以会表现得很好。

当它们在非常不同的取值范围内，如其中一个从1到1000，另一个从0到1，这对优化算法非常不利。但是仅将它们设置为均化零值，假设方差为1，就像上一张幻灯片里设定的那样，确保所有特征都在相似范围内，通常可以帮助学习算法运行得更快。

所以如果输入特征处于不同范围内，可能有些特征值从0到1，有些从1到1000，那么归一化特征值就非常重要了。如果特征值处于相似范围内，那么归一化就不是很重要了。执行这类归一化并不会产生什么危害，我通常会做归一化处理，虽然我不确定它能否提高训练或算法速度。

这就是归一化特征输入，下节课我们将继续讨论提升神经网络训练速度的方法。

1.10 梯度消失/梯度爆炸 (Vanishing / Exploding gradients)

训练神经网络，尤其是深度神经所面临的一个问题就是梯度消失或梯度爆炸，也就是你训练神经网络的时候，导数或坡度有时会变得非常大，或者非常小，甚至于以指数方式变小，这加大了训练的难度。

这节课，你将会了解梯度消失或梯度爆炸的真正含义，以及如何更明智地选择随机初始化权重，从而避免这个问题。假设你正在训练这样一个极深的神经网络，为了节约幻灯片上的空间，我画的神经网络每层只有两个隐藏单元，但它可能含有更多，但这个神经网络会有参数 $W^{[1]}$, $W^{[2]}$, $W^{[3]}$ 等等，直到 $W^{[L]}$ ，为了简单起见，假设我们使用激活函数 $g(z) = z$ ，也就是线性激活函数，我们忽略 b ，假设 $b^{[l]}=0$ ，如果那样的话，输出 $y = W^{[L]} W^{[L-1]} W^{[L-2]} \dots W^{[3]} W^{[2]} W^{[1]} x$ ，如果你想考验我的数学水平， $W^{[1]} x = z^{[1]}$ ，因为 $b = 0$ ，所以我想 $z^{[1]} = W^{[1]} x$ ， $a^{[1]} = g(z^{[1]})$ ，因为我们使用了一个线性激活函数，它等于 $z^{[1]}$ ，所以第一项 $W^{[1]} x = a^{[1]}$ ，通过推理，你会得出 $W^{[2]} W^{[1]} x = a^{[2]}$ ，因为 $a^{[2]} = g(z^{[2]})$ ，还等于 $g(W^{[2]} a^{[1]})$ ，可以用 $W^{[1]} x$ 替换 $a^{[1]}$ ，所以这一项就等于 $a^{[2]}$ ，这个就是 $a^{[3]}(W^{[3]} W^{[2]} W^{[1]} x)$ 。

所有这些矩阵数据传递的协议将给出 $\hat{y}$ 而不是 y 的值。

假设每个权重矩阵 $W^{[l]} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$ ，从技术上来讲，最后一项有不同维度，可能它就是余下的权重矩阵， $y = W^{[1]} \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}^{(L-1)} x$ ，因为我们假设所有矩阵都等于它，它是1.5倍的单位矩阵，最后的计算结果就是 $\hat{y}$ ， $\hat{y}$ 也就是等于 $1.5^{(L-1)} x$ 。如果对于一个深度神经网络来说 L 值较大，那么 $\hat{y}$ 的值也会非常大，实际上它呈指数级增长的，它增长的比率是 1.5^L ，因此对于一个深度神经网络， y 的值将爆炸式增长。

相反的，如果权重是0.5， $W^{[l]} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$ ，它比1小，这项也就变成了 0.5^L ，矩阵 $y = W^{[1]} \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}^{(L-1)} x$ ，再次忽略 $W^{[L]}$ ，因此每个矩阵都小于1，假设 x_1 和 x_2 都是1，激活函数将变成 $\frac{1}{2}$ ， $\frac{1}{2}$ ， $\frac{1}{4}$ ， $\frac{1}{4}$ ， $\frac{1}{8}$ ， $\frac{1}{8}$ 等，直到最后一项变成 $\frac{1}{2^L}$ ，所以作为自定义函数，激活函数的值将以指数级下降，它是与网络层数数量 L 相关的函数，在深度网络中，激活函数以指数级递减。

我希望你得到的直观理解是，权重 W 只比1略大一点，或者说只是比单位矩阵大一点，深度神经网络的激活函数将爆炸式增长，如果 W 比1略小一点，可能是 $\begin{bmatrix} 0.9 & 0 \\ 0 & 0.9 \end{bmatrix}$ 。

在深度神经网络中，激活函数将以指数级递减，虽然我只是讨论了激活函数以与 L 相关的指数级数增长或下降，它也适用于与层数 L 相关的导数或梯度函数，也是呈指数级增长或呈指数递减。

对于当前的神经网络，假设 $L = 150$ ，最近Microsoft对152层神经网络的研究取得了很大进展，在这样一个深度神经网络中，如果激活函数或梯度函数以与 L 相关的指数增长或递减，它们的值将会变得极大或极小，从而导致训练难度上升，尤其是梯度指数小于 L 时，梯度下降算法的步长会非常非常小，梯度下降算法将花费很长时间来学习。

总结一下，我们讲了深度神经网络是如何产生梯度消失或爆炸问题的，实际上，在很长一段时间内，它曾是训练深度神经网络的阻力，虽然有一个不能彻底解决此问题的解决方案，但是已在如何选择初始化权重问题上提供了很多帮助。

1.11 神经网络的权重初始化 (Weight Initialization for Deep Networks Vanishing / Exploding gradients)

上节课，我们学习了深度神经网络如何产生梯度消失和梯度爆炸问题，最终针对该问题，我们想出了一个不完整的解决方案，虽然不能彻底解决问题，却很有用，有助于我们为神经网络更谨慎地选择随机初始化参数，为了更好地理解它，我们先举一个神经单元初始化地例子，然后再演变到整个深度网络。

我们来看看只有一个神经元的情况，然后才是深度网络。

单个神经元可能有4个输入特征，从 x_1 到 x_4 ，经过 $a = g(z)$ 处理，最终得到 $\hat{y}$ ，稍后讲深度网络时，这些输入表示为 $a^{[l]}$ ，暂时我们用 x 表示。

$z = w_1x_1 + w_2x_2 + \dots + w_nx_n$, $b = 0$ ，暂时忽略 b ，为了预防 z 值过大或过小，你可以看到 n 越大，你希望 w_i 越小，因为 z 是 w_ix_i 的和，如果你把很多此类项相加，希望每项值更小，最合理的方法就是设置 $w_i = \frac{1}{n}$ ， n 表示神经元的输入特征数量，实际上，你要做的就是设置某层权重矩阵

$w^{[l]} = np.random.randn(shape) * np.sqrt(\frac{1}{n^{[l-1]}})$ ， $n^{[l-1]}$ 就是我喂给第 l 层神经单元的数量（即第 $l-1$ 层神经元数量）。

结果，如果你是用的是Relu激活函数，而不是 $\frac{1}{n}$ ，方差设置为 $\frac{2}{n}$ ，效果会更好。你常常发现，初始化时，尤其是使用Relu激活函数时， $g^{[l]}(z) = Relu(z)$ ，它取决于你对随机变量的熟悉程度，这是高斯随机变量，然后乘以它的平方根，也就是引用这个方差 $\frac{2}{n}$ 。这里，我用的是 $n^{[l-1]}$ ，因为本例中，逻辑回归的特征是不变的。但一般情况下 l 层上的每个神经元都有 $n^{[l-1]}$ 个输入。如果激活函数的输入特征被零均值和标准方差化，方差是1， z 也会调整到相似范围，这就没解决问题（梯度消失和爆炸问题）。但它确实降低了梯度消失和爆炸问题，因为它给权重矩阵 w 设置了合理值，你也知道，它不能比1大很多，也不能比1小很多，所以梯度没有爆炸或消失过快。

我提到了其它变体函数，刚刚提到的函数是Relu激活函数，一篇由Herd等人撰写的论文曾介绍过。对于几个其它变体函数，如tanh激活函数，有篇论文提到，常量1比常量2的效率更高，对于tanh函数来说，它是 $\sqrt{\frac{1}{n^{[l-1]}}}$ ，这里平方根的作用与这个公式作用相同($np.sqrt(\frac{1}{n^{[l-1]}})$)，它适用于tanh激活函数，被称为Xavier初始化。Yoshua Bengio和他的同事还提出另一种方法，你可能在一些论文中看到过，它们使用的是公式 $\sqrt{\frac{2}{n^{[l-1]} + n^{[l]}}}$ 。其它理论已对此证明，但如果你想用Relu激活函数，也就是最常用的激活函数，我会用这个公式 $np.sqrt(\frac{2}{n^{[l-1]}})$ ，如果使用tanh函数，可以用公式 $\sqrt{\frac{1}{n^{[l-1]}}}$ ，有些作者也会使用这个函数。

实际上,我认为所有这些公式只是给你一个起点,它们给出初始化权重矩阵的方差的默认值,如果你想添加方差,方差参数则是另一个你需要调整的超级参数,可以给公式 $\text{np.sqrt}(\frac{2}{n[l-1]})$ 添加一个乘数参数,调优作为超级参数激增一份子的乘子参数。有时调优该超级参数效果一般,这并不是我想调优的首要超级参数,但我发现调优过程中产生的问题,虽然调优该参数能起到一定作用,但考虑到相比调优,其它超级参数的重要性,我通常把它的优先级放得比较低。

希望你现在对梯度消失或爆炸问题以及如何为权重初始化合理值已经有了一个直观认识,希望你设置的权重矩阵既不会增长过快,也不会太快下降到0,从而训练出一个权重或梯度不会增长或消失过快的深度网络。我们在训练深度网络时,这也是一个加快训练速度的技巧。

1.12 梯度的数值逼近 (Numerical approximation of gradients)

在实施**backprop**时,有一个测试叫做梯度检验,它的作用是确保**backprop**正确实施。因为有时候,你虽然写下了这些方程式,却不能100%确定,执行**backprop**的所有细节都是正确的。为了逐渐实现梯度检验,我们首先说说如何计算梯度的数值逼近,下节课,我们将讨论如何在**backprop**中执行梯度检验,以确保**backprop**正确实施。

我们先画出函数 f ,标记为 $f(\theta)$, $f(\theta) = \theta^3$,先看一下 θ 的值,假设 $\theta = 1$,不增大 θ 的值,而是在 θ 右侧,设置一个 $\theta + \epsilon$,在 θ 左侧,设置 $\theta - \epsilon$ 。因此 $\theta = 1$, $\theta + \epsilon = 1.01$, $\theta - \epsilon = 0.99$,跟以前一样, ϵ 的值为0.01,看下这个小三角形,计算高和宽的比值,就是更准确的梯度预估,选择 f 函数在 $\theta - \epsilon$ 上的这个点,用这个较大三角形的高比上宽,技术上的原因我就不详细解释了,较大三角形的高宽比值更接近于 θ 的导数,把右上角的三角形下移,好像有了两个三角形,右上角有一个,左下角有一个,我们通过这个绿色大三角形同时考虑了这两个小三角形。所以我们得到的不是一个单边公差而是一个双边公差。

我们写一下数据算式,图中绿色三角形上边的点的值是 $f(\theta + \epsilon)$,下边的点是 $f(\theta - \epsilon)$,这个三角形的高度是 $f(\theta + \epsilon) - f(\theta - \epsilon)$,这两个宽度都是 ϵ ,所以三角形的宽度是 2ϵ ,高宽比值为 $\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$,它的期望值接近 $g(\theta)$, $f(\theta) = \theta^3$ 传入参数值, $\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} = \frac{(1.01)^3 - (0.99)^3}{2 \times 0.01}$,大家可以暂停视频,用计算器算算结果,结果应该是3.0001,而前面一张幻灯片上面是,当 $\theta = 1$ 时, $g(\theta) = 3\theta^2 = 3$,所以这两个 $g(\theta)$ 值非常接近,逼近误差为0.0001,前一张幻灯片,我们只考虑了单边公差,即从 θ 到 $\theta + \epsilon$ 之间的误差, $g(\theta)$ 的值为3.0301,逼近误差是0.03,不是0.0001,所以使用双边误差的方法更逼近导数,其结果接近于3,现在我们更加确信, $g(\theta)$ 可能是 f 导数的正确实现,在梯度检验和反向传播中使用该方法时,最终,它与运行两次单边公差的速度一样,实际上,我认为这种方法还是非常值得使用的,因为它的结果更准确。

这是一些你可能比较熟悉的微积分的理论,如果你不太明白我讲的这些理论也没关系,导数的官方定义是针对值很小的 ϵ ,导数的官方定义是 $f'(\theta) = \frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$,如果你上过微积分课,应该学过无穷尽的定义,我就不在这里讲了。

对于一个非零的 ϵ ,它的逼近误差可以写成 $O(\epsilon^2)$, ϵ 值非常小,如果 $\epsilon = 0.01$, $\epsilon^2 = 0.0001$,大写符号 O 的含义是指逼近误差其实是一些常量乘以 ϵ^2 ,但它的确是很准确的逼近误差,所以大写 O 的常量有时是1。然而,如果我们用另外一个公式逼近误差就是 $O(\epsilon)$,当 ϵ 小于1时,实际上 ϵ 比 ϵ^2 大很多,所以这个公式近似值远没有左边公式的准确,所以在执行梯度检验时,我们使用双边误差,即 $\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$,而不使用单边公差,因为它不够准确。

如果你不理解上面两条结论,所有公式都在这儿,不用担心,如果你对微积分和数值逼近有所了解,这些信息已经足够多了,重点是要记住,双边误差公式的结果更准确,下节课我们做梯度检验时就会用到这个方法。

今天我们讲了如何使用双边误差来判断别人给你的函数 $g(\theta)$,是否正确实现了函数 f 的偏导,现在我们可以使用这个方法检验反向传播是否得以正确实施,如果不正确,它可能有bug需要你来解决。

1.13 梯度检验 (Gradient checking)

梯度检验帮我们节省了很多时间，也多次帮我发现**backprop**实施过程中的bug，接下来，我们看看如何利用它来调试或检验**backprop**的实施是否正确。

假设你的网络中含有下列参数， $W^{[1]}$ 和 $b^{[1]}$ $W^{[L]}$ 和 $b^{[L]}$ ，为了执行梯度检验，首先要做的就是，把所有参数转换成一个巨大的向量数据，你要做的就是将矩阵 W 转换成一个向量，把所有 W 矩阵转换成向量之后，做连接运算，得到一个巨型向量 θ ，该向量表示为参数 θ ，代价函数 J 是所有 W 和 b 的函数，现在你得到了一个 θ 的代价函数 J （即 $J(\theta)$ ）。接着，你得到与 W 和 b 顺序相同的数据，你同样可以把 $dW^{[1]}$ 和 $db^{[1]}$ $dW^{[L]}$ 和 $db^{[L]}$ 转换成一个新的向量，用它们来初始化大向量 $d\theta$ ，它与 θ 具有相同维度。

同样的，把 $dW^{[1]}$ 转换成矩阵， $db^{[1]}$ 已经是一个向量了，直到把 $dW^{[L]}$ 转换成矩阵，这样所有的 dW 都已经是矩阵，注意 $dW^{[1]}$ 与 $W^{[1]}$ 具有相同维度， $db^{[1]}$ 与 $b^{[1]}$ 具有相同维度。经过相同的转换和连接运算操作之后，你可以把所有导数转换成一个大向量 $d\theta$ ，它与 θ 具有相同维度，现在的问题是 $d\theta$ 和代价函数 J 的梯度或坡度有什么关系？

这就是实施梯度检验的过程，英语里通常简称为“**grad check**”，首先，我们要清楚 J 是超参数 θ 的一个函数，你也可以将函数展开为 $J(\theta_1, \theta_2, \theta_3, \dots)$ ，不论超级参数向量 θ 的维度是多少，为了实施梯度检验，你要做的就是循环执行，从而对每个 i 也就是对每个 θ 组成元素计算 $d\theta_{\text{approx}}[i]$ 的值，我使用双边误差，也就是

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon}$$

只对 θ_i 增加 ϵ ，其它项保持不变，因为我们使用的是双边误差，对另一边做同样的操作，只不过是减去 ϵ ， θ 其它项全都保持不变。

从上节课中我们了解到这个值（ $d\theta_{\text{approx}}[i]$ ）应该逼近 $d\theta[i] = \frac{\partial J}{\partial \theta_i}$ ， $d\theta[i]$ 是代价函数的偏导数，然后你需要对 i 的每个值都执行这个运算，最后得到两个向量，得到 $d\theta$ 的逼近值 $d\theta_{\text{approx}}$ ，它与 $d\theta$ 具有相同维度，它们两个与 θ 具有相同维度，你要做的就是验证这些向量是否彼此接近。

具体来说，如何定义两个向量是否真的接近彼此？我一般做下列运算，计算这两个向量的距离， $d\theta_{\text{approx}}[i] - d\theta[i]$ 的欧几里得范数，注意这里（ $\|d\theta_{\text{approx}} - d\theta\|_2$ ）没有平方，它是误差平方之和，然后求平方根，得到欧式距离，然后用向量长度归一化，使用向量长度的欧几里得范数。分母只是用于预防这些向量太小或太大，分母使得这个方程式变成比率，我们实际执行这个方程式， ϵ 可能为 10^{-7} ，使用这个取值范围内的 ϵ ，如果你发现计算方程式得到的值为 10^{-7} 或更小，这就很好，这就意味着导数逼近很有可能是正确的，它的值非常小。

如果它的值在 10^{-5} 范围内，我就要小心了，也许这个值没问题，但我会再次检查这个向量的所有项，确保没有一项误差过大，可能这里有**bug**。

如果左边这个方程式结果是 10^{-3} ，我就会担心是否存在**bug**，计算结果应该比 10^{-3} 小很多，如果比 10^{-3} 大很多，我就会很担心，担心是否存在**bug**。这时应该仔细检查所有 θ 项，看是否有一个具体的 i 值，使得 $d\theta_{\text{approx}}[i]$ 与 $d\theta[i]$ 大不相同，并用它来追踪一些求导计算是否正确，经过一些调试，最终结果会是这种非常小的值（ 10^{-7} ），那么，你的实施可能是正确的。

在实施神经网络时，我经常需要执行**foreprop**和**backprop**，然后我可能发现这个梯度检验有一个相对较大的值，我会怀疑存在**bug**，然后开始调试，调试，调试，调试一段时间后，我得到一个很小的梯度检验值，现在我可以很自信的说，神经网络实施是正确的。

现在你已经了解了梯度检验的工作原理，它帮助我在神经网络实施中发现了很多**bug**，希望它对你也有所帮助。

1.14 梯度检验应用的注意事项（Gradient Checking Implementation Notes）

这节课，分享一些关于如何在神经网络实施梯度检验的实用技巧和注意事项。

首先，不要在训练中使用梯度检验，它只用于调试。我的意思是，计算所有 i 值的 $d\theta_{\text{approx}}[i]$ 是一个非常漫长的计算过程，为了实施梯度下降，你必须使用 W 和 b **backprop**来计算 $d\theta$ ，并使用**backprop**来计算导数，只要调试的时候，你才会计算它，来确认数值是否接近 $d\theta$ 。完成后，你会关闭梯度检验，梯度检验的每一个迭代过程都不执行它，因为它太慢了。

第二点，如果算法的梯度检验失败，要检查所有项，检查每一项，并试着找出**bug**，也就是说，如果 $d\theta_{\text{approx}}[i]$ 与 $d\theta[i]$ 的值相差很大，我们要做的就是查找不同的 i 值，看看是哪个导致 $d\theta_{\text{approx}}[i]$ 与 $d\theta[i]$ 的值相差这么多。举个例子，如果你发现，相对某些层或某层的 θ 或 $d\theta$ 的值相差很大，但是 $dw^{[l]}$ 的各项非常接近，注意 θ 的各项与 b 和 w 的各项都是一一对应的，这时，你可能会发现，在计算参数 b 的导数 db 的过程中存在**bug**。反过来也是一样，如果你发现它们的值相差很大， $d\theta_{\text{approx}}[i]$ 的值与 $d\theta[i]$ 的值相差很大，你会发现所有这些项目都来自于 dw 或某层的 dw ，可能帮你定位bug的位置，虽然未必能够帮你准确定位bug的位置，但它可以帮助你估测需要在哪些地方追踪**bug**。

第三点，在实施梯度检验时，如果使用正则化，请注意正则项。如果代价函数 $J(\theta) = \frac{1}{m} \sum L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum \|W^{[l]}\|^2$ ，这就是代价函数 J 的定义， $d\theta$ 等于与 θ 相关的 J 函数的梯度，包括这个正则项，记住一定要包括这个正则项。

第四点，梯度检验不能与**dropout**同时使用，因为每次迭代过程中，**dropout**会随机消除隐藏层单元的不同子集，难以计算**dropout**在梯度下降上的代价函数 J 。因此**dropout**可作为优化代价函数 J 的一种方法，但是代价函数 J 被定义为对所有指数极大的节点子集求和。而在任何迭代过程中，这些节点都有可能被消除，所以很难计算代价函数 J 。你只是对成本函数做抽样，用**dropout**，每次随机消除不同的子集，所以很难用梯度检验来双重检验**dropout**的计算，所以我一般不同时使用梯度检验和**dropout**。如果你想这样做，可以把**dropout**中的**keepprob**设置为1.0，然后打开**dropout**，并寄希望于**dropout**的实施是正确的，你还可以做点别的，比如修改节点丢失模式确定梯度检验是正确的。实际上，我一般不这么做，我建议关闭**dropout**，用梯度检验进行双重检查，在没有**dropout**的情况下，你的算法至少是正确的，然后打开**dropout**。

最后一点，也是比较微妙的一点，现实中几乎不会出现这种情况。当 w 和 b 接近0时，梯度下降的实施是正确的，在随机初始化过程中.....，但是在运行梯度下降时， w 和 b 变得更大。可能只有在 w 和 b 接近0时，**backprop**的实施才是正确的。但是当 W 和 b 变大时，它会变得越来越不准确。你需要做一件事，我不经常这么做，就是在随机初始化过程中，运行梯度检验，然后再训练网络， w 和 b 会有一段时间远离0，如果随机初始化值比较小，反复训练网络之后，再重新运行梯度检验。

这就是梯度检验，恭喜大家，这是本周最后一课了。回顾这一周，我们讲了如何配置训练集，验证集和测试集，如何分析偏差和方差，如何处理高偏差或高方差以及高偏差和高方差并存的问题，如何在神经网络中应用不同形式的正则化，如 $L2$ 正则化和**dropout**，还有加快神经网络训练速度的技巧，最后是梯度检验。这一周我们学习了很多内容，你可以在本周编程作业中多多练习这些概念。祝你好运，期待下周再见。